

Методические указания
для выполнения практической работы №1
“Системы контроля версий”

По дисциплине «Технологии разработки программных приложений»

Составители методички: Воронцов Ю.А., Михайлова Е.К.

Составители задания: Торхов А.Е., Копылова А.В.

Оглавление

Задание	2
Часть 1. Основные команды Git	3
Часть 2. Системы управления репозиториями	3
Часть 3. Работа с ветвлением и оформление кода	6
Часть 1. Как начать работу с git’ом?	8
Шаг 1. Устанавливаем Git на локальную машину	8
Установка в Linux и Unix	8
Установка на macOS	8
Установка в Windows	8
Проверим, что Git установлен	8
Шаг 2. Настройка Git	8
Шаг 3. Начало работы – создание папки и файлов	9
Шаг 4. Создание репозитория	11
Шаг 5. Добавление файла в репозиторий и добавление первого коммита	11
Шаг 6. Индексация изменений	12
Шаг 6*. Индексация и коммит (не обязательный) – для ознакомления	12
Шаг 7. Коммиты нескольких изменений	13
Шаг 8. Просмотр истории коммитов	15
Шаг 9. Получение старых версий	17
Шаг 10. Отмена локальных изменений (до индексации)	18
Шаг 11. Отмена локальных изменений (после индексации и до коммита)	19
Шаг 12. Отмена коммита	20
Часть 2. Управление репозиториями	21
Шаг 1. Создание аккаунта на GitHub	21
Шаг 2. Создание SSH-ключа для авторизации	26
Шаг 3. Создание нового репозитория для своего проекта	30
Шаг 4. Связываем локальный и удаленный репозитории	30
Шаг 5. Создание веток и переключение между ними	32

Шаг 6. Слияние веток	32
Дополнительная информация для выполнения заданий по вариантам	34
1. Клонирование удаленного репозитория	34
2. Выгрузка изменений в удаленный репозиторий	35
3. Вывод в консоль различий между веткой мастер и другой веткой	35
4. Создание тэгов к коммитам	35
5. Работа с отложенными изменениями - git stash	37
6. Перемещение коммита в другую ветку	38
8. Вывод изменений строк файла по коммитам	39
Часть 3. Ветвление и оформление кода	41
Шаг 1. Форк репозитория	41
Шаг 2. Клонирование репозитория на локальную машину	41
Шаг 3. Создание двух веток в master'e	41
Шаг 4. 3 коммита в каждую из веток, которые меняют один и тот же кусочек файла	41
Шаг 5. Слияние ветки branch1 в ветку branch2 с разрешением конфликтов	41
Шаг 6. Выгрузка всех изменений во всех ветках в удаленный репозиторий	42
Шаг 7. Выполнение 3-х коммитов в branch1	42
Шаг 8. Повторное клонирование репозитория в другую директорию	42
Шаг 9. Выполнение 3-х коммитов в ветку branch1	42
Шаг 10. Выгрузка всех изменений в удаленный репозиторий	42
Шаг 11. Выгрузка изменения из старого репозитория с опцией --force	43
Шаг 12. Получение всех изменения в новом репозитории	43
Примечания	44
Где находится файл .gitconfig в macOS?	44
Что делать, если возникает ошибка при git init?	44
Контрольные вопросы	45
Требования к отчету	46

Задание

Цель работы: получить навыки по работе с командной строкой и git'ом.

Часть 1. Основные команды Git

1. Установите и настройте клиент git на своей рабочей станции.
2. Создайте локальный репозиторий и добавьте в него несколько файлов.
3. Внесите изменения в один из файлов.
4. Проиндексируйте изменения и проверьте состояние.
5. Сделайте коммит того, что было проиндексировано в репозиторий. Добавьте к коммиту комментарий.
6. Измените еще один файл. Добавьте это изменение в индекс git. Измените файл еще раз. Проверьте состояние и произведите коммит проиндексированного изменения. Теперь добавьте второе изменение в индекс, а затем проверьте состояние с помощью команды `git status`. Сделайте коммит второго изменения.
7. Просмотрите историю коммитов с помощью команды `git log`. Ознакомьтесь с параметрами команды и используйте некоторые из них для различного формата отображения истории коммитов.
8. Верните рабочий каталог к одному из предыдущих состояний.
9. Изучите, как создавать теги для коммитов для использования в будущем.
10. Отмените некоторые изменения в рабочем каталоге (до и после индексирования).
11. Отмените один из коммитов в локальном репозитории.

Часть 2. Системы управления репозиториями

1. Создайте аккаунт на GitHub (у кого нет),
2. Создайте репозиторий на GitHub и на локальной машине, согласно выбранной теме проекта,
3. Создайте несколько файлов на локальной машине при помощи консоли,
4. Создайте SSH-ключ для авторизации,
5. Свяжите репозиторий локальной машины с репозиторием на GitHub при помощи консоли,
6. Создайте новую ветку в репозитории с помощью команды, произведите в ней какие-нибудь изменения, а после слейте с веткой master,

7. Выполните цепочку действий в репозитории, согласно вариантам.

№ варианта	Задание
1	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать новую ветку и вывести список всех веток 3) Произвести 3 коммита в новой ветке 4) Выгрузить изменения в удаленный репозиторий 5) Откатить ветку обратно на 2 коммита (в том числе в удаленном репозитории) 6) Вывести в консоли различия между веткой master и новой веткой 7) Слить новую ветку с master при помощи merge
2	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать новую ветку и вывести список всех веток 3) Произвести коммит в ветке master 4) Произвести 3 коммита в новой ветке в разные файлы 5) Выгрузить изменения в удаленный репозиторий 6) Откатить ветку обратно на 2 коммита (в том числе в удаленном репозитории) 7) Вывести в консоли различия между веткой master и новой веткой 8) Перебазировать новую ветку на master
3	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать тег указывающий на последний коммит в ветке master 3) Создать новую ветку и вывести список всех веток 4) Произвести 3 коммита в новой ветке 5) Выгрузить все изменения в удаленный репозиторий 6) Откатить ветку к созданному тегу (в том числе в удаленном репозитории) 7) Вывести в консоли различия между веткой master и новой веткой
4	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать новую ветку и вывести список всех веток 3) Произвести 3 коммита в новой ветке в разные файлы 4) Выгрузить изменения в удаленный репозиторий 5) Откатить в новой ветке предпоследний коммит (в том числе в удаленном репозитории) 6) Вывести в консоли различия между веткой master и новой веткой 7) Слить новую ветку с master при помощи merge

5	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать новую ветку и вывести список всех веток 3) Произвести 3 коммита в новой ветке в разные файлы 4) Выгрузить изменения в удаленный репозиторий 5) Произвести revert предпоследнего коммита в новой ветке 6) Вывести в консоли различия между веткой master и новой веткой 7) Слить новую ветку с master при помощи merge
6	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать новую ветку и вывести список всех веток 3) Произвести 3 коммита в новой ветке в новых файлах 4) Выгрузить изменения в удаленный репозиторий 5) Произвести какие-нибудь изменения в файле который существует в ветке master, но не коммитьте их 6) Выполните git stash 7) Переключитесь на ветку master 8) Выполните git stash pop 9) Выведите в консоли различия между веткой master и новой веткой 10) Слить новую ветку с master при помощи merge
7	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать две новых ветки и вывести список всех веток 3) Произвести 3 коммита в первой ветке 4) Переместите (cherry pick) первый коммит во вторую ветку 5) Выгрузить изменения в удаленный репозиторий 6) Выведите в консоли различия между веткой master и новыми ветками 7) Слить обе ветки с master при помощи merge
8	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать две новых ветки и вывести список всех веток 3) Произвести по 2 коммита в обеих ветках в новых файлах 4) Выгрузить изменения в удаленный репозиторий 6) Выведите в консоли различия между веткой master и обоими ветками 7) Слить одну ветку с master при помощи merge 8) Подлейте master ко второй ветке 9) Выведите в консоли различия между веткой master и обоими ветками

9	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать новую ветку и вывести список всех веток 3) Произвести 5 коммитов в новой ветке в разные файлы 4) Выгрузить изменения в удаленный репозиторий 5) Произвести revert 2-го и 4-го коммита в новой ветке 6) Вывести в консоли различия между веткой master и новой веткой 7) Слить новую ветку с master при помощи merge
10	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать новую ветку и вывести список всех веток 3) Произвести 3 коммита в новой ветке в разные файлы 4) Выгрузить изменения в удаленный репозиторий 5) Произвести какие-нибудь изменения в файле который существует в ветке, но не коммитьте их 6) Внесите эти изменения в последний коммит (amend) 7) Выведите в консоли различия между веткой master и новой веткой 8) Слить новую ветку с master при помощи merge
11	1) Клонировать непустой удаленный репозиторий на локальную машину 2) Создать новую ветку и вывести список всех веток 3) Произвести 3 коммита в новой ветке в разные файлы 4) Выгрузить изменения в удаленный репозиторий 5) Добейтесь того, чтобы эти три изменения были представлены одним коммитом 6) Выведите в консоли различия между веткой master и новой веткой 7) Слить новую ветку с master при помощи merge

Часть 3. Работа с ветвлением и оформление кода

1. Сделайте форк репозитория в соответствии с вашим вариантом
2. Склонируйте его на локальную машину
3. Создайте две ветки `branch1` и `branch2` от последнего коммита в `master`'е
4. Проведите по 3 коммита в каждую из веток, которые меняют один и тот же кусочек файла
5. Выполните слияние ветки `branch1` в ветку `branch2`, разрешив конфликты при этом
6. Выгрузите все изменения во всех ветках в удаленный репозиторий
7. Проведите еще 3 коммита в ветку `branch1`
8. Склонируйте репозиторий еще раз в другую директорию
9. В новом клоне репозитории сделайте 3 коммита в ветку `branch1`
10. Выгрузите все изменения из нового репозитория в удаленный репозиторий
11. Вернитесь в старый клон с репозиторием, выгрузите изменения с опцией `--force`
12. Получите все изменения в новом репозитории

Варианты:

№ варианта	Репозиторий
1	https://github.com/liamg/traitor
2	https://github.com/google/model_search
3	https://github.com/gto76/python-cheatsheet
4	https://github.com/ripienaar/free-for-dev
5	https://github.com/pystardust/ytfzf
6	https://github.com/airbnb/javascript
7	https://github.com/kettanaito/naming-cheatsheet
8	https://github.com/Sairyss/domain-driven-hexagon
9	https://github.com/react-native-camera/react-native-camera
10	https://github.com/spring-projects/spring-petclinic

Часть 1. Как начать работу с git'ом?

Шаг 1. Устанавливаем Git на локальную машину

Установка в Linux и Unix

- Используйте обычный менеджер пакетов вашего дистрибутива. Откройте терминал и введите подходящие команды.
- Если у вас 21 или более ранняя версия Fedora, используйте `yum install git`.
- Для 22 и последующих версий Fedora вводите `dnf install git`.
- Для дистрибутивов, основанных на Debian, например, Ubuntu, используйте `apt-get: sudo apt-get install git`.

Полный список команд для различных дистрибутивов можно посмотреть [здесь](#).

Установка на macOS

- Скачиваем Git со страницы [проекта](#).
- Запускаем загруженный файл.
- Система может показать окно с ошибкой, где будет написано, что файл скачан с неавторизованного сайта и инсталлятор не может быть запущен. В таком случае нужно зайти в «Системные настройки» — «Безопасность» (Security and Privacy), в появившемся окне будет сообщение об ошибке и кнопка Open anyway (Всё равно открыть). Нажимаем.
- Система покажет окно, уточняющее хотите ли вы запустить установку. Подтверждаем действие.
- Установщик проведёт через все необходимые шаги.

Установка в Windows

Скачайте exe-файл инсталлятора с [сайта Git](#) и запустите его. Это Git для Windows, он называется msysGit. Установщик спросит добавлять ли в меню проводника возможность запуска файлов с помощью Git Bash (консольная версия) и GUI (графическая версия). Подтвердите действие, чтобы далее вести работу через консоль в Git Bash. Остальные пункты можно оставить по умолчанию.

Проверим, что Git установлен

После того, как все действия по установке завершены, убедимся, что Git появился в системе компьютера. Откройте терминал и введите `git --version`, должна появиться текущая версия программы на вашей машине. Эта проверка подходит для всех операционных систем.

Шаг 2. Настройка Git

1. Открываем терминал
2. Необходимо выполнить следующие команды:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@whatever.com"
```

где **You Name** – ваш логин, **your_email@whatever.com** – ваша почта

3. Необходимо выполнить следующие команды:

Для пользователей Unix/Mac:

```
git config --global core.autocrlf input
git config --global core.safecrlf warn
```

Для пользователей Windows:

```
git config --global core.autocrlf true
git config --global core.safecrlf warn
```

4. Необходимо выполнить следующую команду:

```
git config --global core.quotepath off
```

Для проверки верности всех введенных команд, введите:

```
git config --list
```

```
user.name=Evgenia
user.email=michaylova1996@mail.ru
core.autocrlf=input
core.safecrlf=warn
core.quotepath=off
```

Если получилось все, как на картинке (с вашей парой логин и почта), то все верно.

Шаг 3. Начало работы – создание папки и файлов

1. Для перехода в другую директорию воспользуйтесь командой `cd <путь к директории>`. Например:

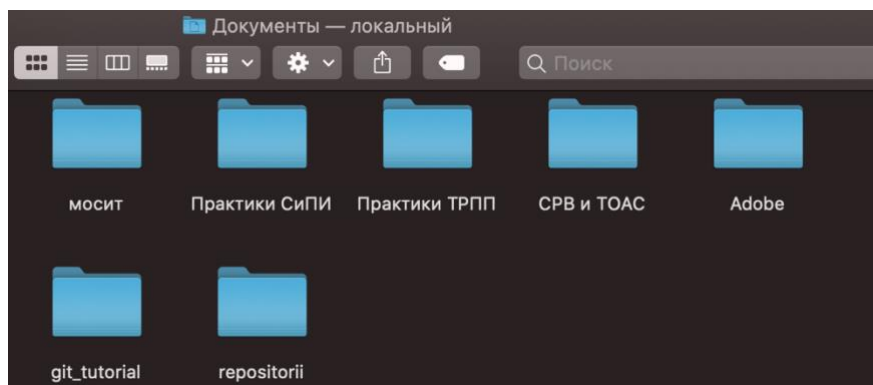
```
Evgenia ~ % cd Documents
Evgenia Documents %
```

Примечание: в Git bash необходимо прописывать полный путь и экранировать его двойным слэшем. Например: C:\\Users\\User\\Documents

2. Создаем папку, в которой будет лежать наш репозиторий с помощью команды `mkdir <Название папки>`, например:

```
Evgenia Documents % mkdir repositorii
```

В результате данной команды, по вашему пути будет создана папка



3. Перейдите в созданную папку с помощью команды `cd`

```
Documents % cd repositorii
```

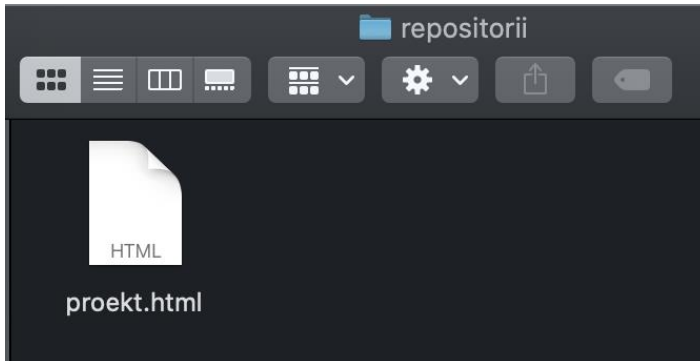
Результатом выполнения будет переход в созданную директорию:

```
Evgenia repositorii %
```

4. Создаем файл с помощью команды ***touch*** *<имя файла>* с расширением ***.html*** – пример на скриншоте ниже

```
Evgenia repositorii % touch proekt.html
```

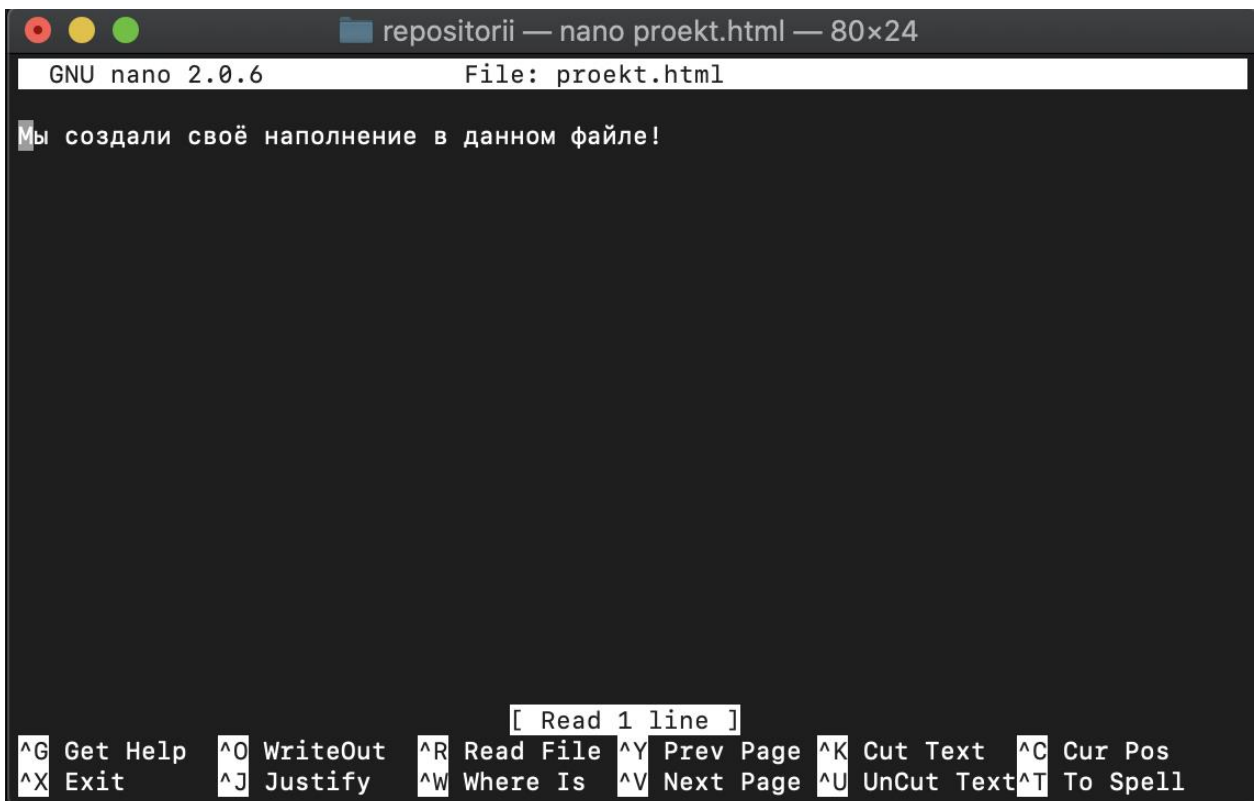
Результатом выполнения данной команды будет созданный файл:



5. Для добавления в файл содержания, можно воспользоваться командой ***nano***, которая вызовет редактор nano. Можно использовать редактор vim (но только понимающим и разбирающимся пользователями).

```
Evgenia repositorii % nano proekt.html
```

В результате данного действия откроется окно с редактором текста в консоли.



Вписываем свой текст в данный файл и нажимаем ctrl+x, для сохранения нажимаем Y, а далее Enter.

Шаг 4. Создание репозитория

1. Выполняем команду: `git init`

```
Evgenia repositorii % git init
```

После выполнения данной команды, должно высветиться данное сообщение:

```
Инициализирован пустой репозиторий Git в /Users/xkaprizx/Documents/repositorii/.git/
```

Шаг 5. Добавление файла в репозиторий и добавление первого коммита

Чтобы добавить файл в репозиторий необходимо выполнить следующее:

1. Введите команды:

```
git add <Название вашего файла>
```

```
git commit -m "Ваш текст для коммита"
```

Примечание: при выполнении команды с коммитом кавычки обязательны, в названии же файла их быть не должно.

Например:

```
Evgenia repositorii % git commit -m "Наш первый коммит"
```

В результате мы должны получить следующее:

```
[master (корневой коммит) 09925b9] Наш первый коммит
1 file changed, 1 insertion(+)
create mode 100644 proekt.html
```

2. Проверка состояния репозитория

Выполняем команду:

```
git status
```

Мы получим следующее сообщение:

```
На ветке master
ничего коммитить, нет изменений в рабочем каталоге
```

Команда проверки состояния сообщит, что коммитить нечего. Это означает, что в репозитории хранится текущее состояние рабочего каталога, и нет никаких изменений, ожидающих записи.

3. Внесение изменений в файл.

Снова воспользуемся командой nano, как в шаге 4.

```
repositorii % nano proekt.html
```

Не забываем сохранять изменения в файле при помощи `ctrl+x`, `Y`, `Enter`.

Выполняем команду из шага 5.

```
Evgenia repositorii % git status
```

Получаем результат:

```
На ветке master
Изменения, которые не в индексе для коммита:
  (используйте «git add <файл>...», чтобы добавить файл в индекс)
  (use "git restore <file>..." to discard changes in working directory)
       изменено:      proekt.html

нет изменений добавленных для коммита
(используйте «git add» и/или «git commit -a»)
```

В данной ситуации `git` распознал, что ваш файл был изменен, но эти изменения не зафиксированы в репозитории.

`Git` нам подсказывает, что можно добавить эти изменения в репозиторий и показывает какую команду необходимо для этого использовать.

Шаг 6. Индексация изменений

Для индексации изменений необходимо выполнить команды:

```
git add <Имя вашего файла>
git status
```

```
Evgenia repositorii % git add proekt.html
Evgenia repositorii % git status
```

Результат должен получиться:

```
На ветке master
Изменения, которые будут включены в коммит:
  (use "git restore --staged <file>..." to unstage)
       изменено:      proekt.html
```

Теперь `Git` знает об изменении.

Шаг 6*. Индексация и коммит (не обязательный) – для ознакомления

Создадим в директории еще 3 файла при помощи команды `touch <имя файла>`

```
Evgenia repositorii % touch a.html
Evgenia repositorii % touch b.html
Evgenia repositorii % touch c.html
```

Отдельный шаг индексации в git позволяет вам продолжать вносить изменения в рабочий каталог, а затем, в момент, когда вы захотите взаимодействовать с версионным контролем, git позволит записать изменения в малых коммитах, которые фиксируют то, что вы сделали.

Предположим, что вы отредактировали три файла (a.html, b.html, и c.html). Теперь вы хотите закомитить все изменения, при этом чтобы изменения в a.html и b.html были одним коммитом, в то время как изменения в c.html логически не связаны с первыми двумя файлами и должны идти отдельным коммитом.

Введите следующие команды:

```
git add a.html
git add b.html
git commit -m "Changes for a and b"
```

```
git add c.html
git commit -m "Unrelated change to c"
```

Разделяя индексацию и коммит, вы имеете возможность с легкостью настроить, что идет в какой коммит.

Шаг 7. Коммиты нескольких изменений

1. Внесение изменений в файл.

Внесите изменения в свой файл с помощью команды nano:

```
Evgenia repositorii % nano proekt.html
```

```
<html>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>
```

Выполните команду:

```
git add «Название файла»
```

Внесите второе изменение в файл:

```
<html>

  <head>
  </head>
  <body>
    <h1>Hello, World!</h1>
  </body>
```

</html>

2. Проверка статуса репозитория

Выполните команду:

```
git status
```

Результат выполнения:

```
На ветке master
Изменения, которые будут включены в коммит:
  (use "git restore --staged <file>..." to unstage)
        изменено:   projekt.html

Изменения, которые не в индексе для коммита:
  (используйте «git add <файл>...», чтобы добавить файл в индекс)
  (use "git restore <file>..." to discard changes in working directory)
        изменено:   projekt.html
```

3. Добавление коммита

Давайте добавим коммит при помощи следующей команды и проверим статус репозитория:

```
git commit -m "Added standard HTML page tags"
```

```
git status
```

```
Evgenia repositorii % git commit -m "Added standart HTML page tags"
Evgenia repositorii % git status
```

Результат выполнения:

```
На ветке master
Изменения, которые не в индексе для коммита:
  (используйте «git add <файл>...», чтобы добавить файл в индекс)
  (use "git restore <file>..." to discard changes in working directory)
        изменено:   projekt.html

Неотслеживаемые файлы:
  (используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)
[
        .DS_Store

нет изменений добавленных для коммита
(используйте «git add» и/или «git commit -a»)
```

Примечание: В операционной системе MacOS присутствует дополнительный системный файл `.DS_Store`, который будет отображен как не отслеживаемый в выводе `git status`.

Выполняем следующие команды:

```
git add .  
git status
```

Результат выполнения:

```
На ветке master  
Изменения, которые будут включены в коммит:  
(use "git restore --staged <file>..." to unstage)  
    новый файл:      .DS_Store  
    изменено:       proekt.html
```

После этого делаем коммит второго изменения.

```
git commit -m "Added HTML header"
```

Шаг 8. Просмотр истории коммитов

Для просмотра истории коммитов используется команда:

```
git log
```

Результат выполнения:

```
commit bbee4102c21c0a946eadb270e27eb1e5649e80d3 (HEAD -> master)  
Author: Evgenia <michaylova1996@mail.ru>  
Date:   Fri Feb 12 10:40:30 2021 +0300  
  
    Added standart HTML page tags  
  
commit f90f26711a4a43248cf2ee2a23d5ae334d6a8cc6  
Author: Evgenia <michaylova1996@mail.ru>  
Date:   Fri Feb 12 10:15:25 2021 +0300  
  
    Unrelated change to c  
  
commit 8db4b7f9e761c80d099dac8b3445899b5c537a2f  
Author: Evgenia <michaylova1996@mail.ru>  
Date:   Fri Feb 12 10:14:43 2021 +0300  
  
    Changes for a and b)  
  
commit 09925b9b0da5abd2f784c5b506b43e8f5b2592a4  
Author: Evgenia <michaylova1996@mail.ru>  
Date:   Fri Feb 12 09:47:12 2021 +0300  
  
    Наш первый коммит
```

Команда **log** позволяет контролировать формат выводимой информации:

```
git log --pretty=oneline
```


Результат выполнения:

```
bbee4102c21c0a946eadb270e27eb1e5649e80d3 (HEAD -> master) Added standart HTML page tags
f90f26711a4a43248cf2ee2a23d5ae334d6a8cc6 Unrelated change to c
8db4b7f9e761c80d099dac8b3445899b5c537a2f Changes for a and b)
09925b9b0da5abd2f784c5b506b43e8f5b2592a4 Наш первый коммит
```

Для контроля отображения записей можно использовать следующие параметры:

```
git log --pretty=oneline --max-count=2
git log --pretty=oneline --since='5 minutes ago'
git log --pretty=oneline --until='5 minutes ago'
git log --pretty=oneline --author=<your name>
git log --pretty=oneline --all
```

Попробуйте использовать эти команды и поэкспериментируйте с выводом при помощи них.

В unix-системах доступна справочная страница `man git log`

Для уточнения формата лога используется флаг `--pretty=format:"..."`

Для примера выполните команду:

```
git log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short
```

Рассмотрим её в деталях:

- `%h` — укороченный хэш коммита
- `%d` — дополнения коммита («головы» веток или теги)
- `%ad` — дата коммита
- `%s` — комментарий
- `%an` — имя автора
- `--graph` — отображает дерево коммитов в виде ASCII-графика
- `--date=short` — сохраняет формат даты коротким

Результат выполнения:

```
* bbee410 2021-02-12 | Added standart HTML page tags (HEAD -> master) [Evgenia]
* f90f267 2021-02-12 | Unrelated change to c [Evgenia]
* 8db4b7f 2021-02-12 | Changes for a and b) [Evgenia]
* 09925b9 2021-02-12 | Наш первый коммит [Evgenia]
```


Шаг 9. Получение старых версий

1. Для получения старой версии нашего каталога необходимо узнать для начала хэши, для этого воспользуемся командой из предыдущего шага:

```
git log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short
```

```
* c993c20 2021-02-15 | pervii commit na GitHub (HEAD -> master, origin/master) [Евгения]
* bbee410 2021-02-12 | Added standart HTML page tags (newbranch) [Evgenia]
* f90f267 2021-02-12 | Unrelated change to c [Evgenia]
* 8db4b7f 2021-02-12 | Changes for a and b) [Evgenia]
* 09925b9 2021-02-12 | Наш первый коммит [Evgenia]
```

Как видно со скрина, первый коммит идет с хэшем 09925b9 (у вас он будет абсолютно другой, используйте его).

Введем команду:

```
Evgenia repositorii % git checkout 09925b9
```

Результатом будет:

```
Note: switching to '09925b9'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD сейчас на 09925b9 Наш первый коммит
```

Проверим, что в данный момент находится в файле с помощью команды cat:

```
Evgenia repositorii % cat proekt.html
```

В результате мы видим, что наполнение файла изменилось на самые первоначальные данные в этом файле:

```
Evgenia repositorii % cat proekt.html
Мы создали своё наполнение в данном файле!
```

Вернемся в нынешнее последнее состояние при помощи команды git checkout и проверим, что находится в файле с помощью команды cat:

```
Evgenia repositorii % git checkout master
```

```
Предыдущая позиция HEAD была 09925b9 Наш первый коммит
Переключено на ветку «master»
```

```
Evgenia repositorii % cat proekt.html
```

```
<html>
<head>
</head>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
</body>
</html>
```

Шаг 10. Отмена локальных изменений (до индексации)

1. Убедимся, что мы находимся на последнем коммите ветки master, прежде чем продолжить работу, для этого введем команду:

```
git checkout master
```

2. Изменим наш файл в рабочем каталоге при помощи команды *nano <имя файла>* и сохраним его:

```
-Evgenia repositorii % nano projekt.html
```

```
<html>
<head>
</head>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
<!-- Это плохой комментарий. Мы должны убрать его. -->
</body>
</html>
```

3. Проверим состояние рабочего каталога при помощи команды *git status*:

```
-Evgenia repositorii % git status
```

В результате нашей проверки получим следующее:

```
На ветке master
Изменения, которые не в индексе для коммита:
  (используйте «git add <файл>...», чтобы добавить файл в индекс)
  (use "git restore <file>..." to discard changes in working directory)
       изменено:   projekt.html

нет изменений добавленных для коммита
(используйте «git add» и/или «git commit -a»)
```

Это указывает на то, что файл был изменен, но еще не был проиндексирован

4. Для отмены изменений воспользуемся следующими командами и проверим, что изменилось:

```
git checkout <имя файла>
```

```
Updated 1 path from the index
```

```
git status
```

```
На ветке master
ничего коммитить, нет изменений в рабочем каталоге
```

```
cat <имя файла>
```

```
<html>
<head>
</head>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
</body>
</html>
```

Шаг 11. Отмена локальных изменений (после индексации и до коммита)

1. Внесем снова изменения в наш файл при помощи команды `nano <имя файла>`:

```
<html>
<head>
<!-- Это нежелательный комментарий! -->
</head>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
</body>
</html>
```

2. Проиндексируем наши изменения с помощью команды `git add <имя файла>`:

```
Evgenia repositorii % git add projekt.html
```

3. Проверим состояние нашего изменения при помощи команды `git status`:

```
Evgenia repositorii % git status
```

В результате получим сообщение о том, что изменение было готово к коммиту:

```
На ветке master
Изменения, которые будут включены в коммит:
  (use "git restore --staged <file>..." to unstage)
        изменено:   projekt.html
```

4. Отменим индексацию изменений:

```
git reset HEAD <имя файла>
```

```
Evgenia repositorii % git reset HEAD projekt.html
```

В результате мы получим следующее сообщение:

```
Непроиндексированные изменения после сброса:
M       projekt.html
```

Команда `reset` сбрасывает буферную зону к HEAD. Это очищает буферную зону от изменений, которые мы только что проиндексировали.

Команда `reset` (по умолчанию) не изменяет рабочий каталог. Поэтому рабочий каталог все еще содержит нежелательный комментарий. Поэтому мы воспользуемся командой `checkout` из прошлого шага, чтобы убрать нежелательный комментарий.

5. Убираем нежелательный комментарий при помощи команды `git checkout <имя файла>`:

```
Evgenia repositorii % git checkout projekt.html
```

В результате мы получаем такое сообщение:

```
Updated 1 path from the index
```

6. Проверяем статус рабочего каталога при помощи команды `git status`:

```
Evgenia repositorii % git status
```

Получаем сообщение, что нет изменений в рабочем каталоге, а значит нежелательного изменения в файле больше нет:

```
На ветке master  
нечего коммитить, нет изменений в рабочем каталоге
```

Шаг 12. Отмена коммита

1. Внесем снова изменения в наш файл при помощи команды `nano <имя файла>`:

```
<html>  
<head>  
<!-- Это нежелательный комментарий! -->  
</head>  
<body>  
<h1>Мы создали своё наполнение в данном файле!</h1>  
</body>  
</html>
```

2. Проиндексируем наши изменения с помощью команды `git add <имя файла>`:

```
Evgenia repositorii % git add proekt.html
```

3. Произведем следующий коммит при помощи команды `git commit -m "Название коммита"`:

```
Evgenia repositorii % git commit -m "Ой, нам не нужен данный коммит"
```

Примечание. В следующем пункте открывается редактор, который установлен по умолчанию, в большинстве случаев это редактор vi или vim не дружелюбный к новичкам, поэтому мы советуем прописать глобально редактор nano, которым вы пользуетесь с самого начала данной методички, для этого необходимо прописать в командную строку следующую команду: `git config --global core.editor nano`

4. Чтобы отменить коммит, нам необходимо сделать коммит, который удаляет изменения, сохраненные нежелательным коммитом, для это воспользуемся следующей командой:

```
git revert HEAD
```

После выполнения данной команды откроется редактор, который предложит вам отредактировать коммит или вы можете оставить его как есть. Можно также воспользоваться параметром `--no-edit`

```
Evgenia repositorii % git revert HEAD --no-edit
```

Результатом данной команды будет:

```
[master de0643d] Revert "Ой, нам не нужен данный коммит"
Date: Wed Mar 3 18:51:37 2021 +0300
1 file changed, 1 deletion(-)
```

5. Проверим историю коммитов при помощи команды `git log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short`

В результате мы увидим следующее:

```
* de0643d 2021-03-03 | Revert "Ой, нам не нужен данный коммит" (HEAD -> master) [Евгения]
* f6d003e 2021-03-03 | Ой, нам не нужен данный коммит [Евгения]
* c993c20 2021-02-15 | pervii commit na GitHub (origin/master) [Евгения]
* bbee410 2021-02-12 | Added standart HTML page tags (newbranch) [Evgenia]
* f90f267 2021-02-12 | Unrelated change to c [Evgenia]
* 8db4b7f 2021-02-12 | Changes for a and b) [Evgenia]
* 09925b9 2021-02-12 | Наш первый коммит [Evgenia]
```

Часть 2. Управление репозиториями

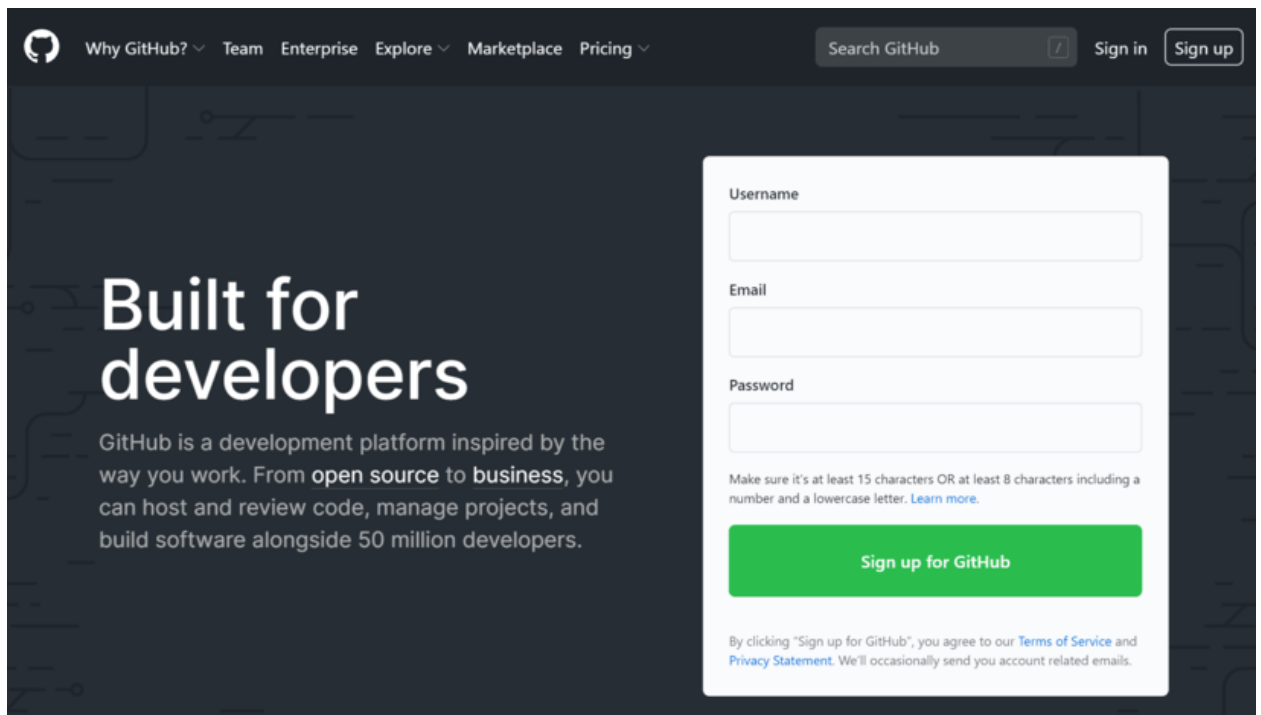
Шаг 1. Создание аккаунта на GitHub

Что такое GitHub?

GitHub — веб-сервис, который основан на системе Git. Это такая социальная сеть для разработчиков, которая помогает удобно вести коллективную разработку IT-проектов. Здесь можно публиковать и редактировать свой код, комментировать чужие наработки, следить за новостями других пользователей. Именно в GitHub работаем мы, команда Академии, и студенты интенсивов.

Чтобы начать работу с GitHub, нужно зарегистрироваться на сайте, если вы ещё этого не сделали.

1. Переходим на [сайт GitHub](#).



Стартовая страница GitHub.

Для начала регистрации:

- Нажимаем кнопку Sign up (зарегистрироваться), попадаем на страницу регистрации, где вводим обязательные данные: имя пользователя, адрес электронной почты и пароль. После заполнения полей проходим верификацию.

The screenshot shows the GitHub registration page. At the top is a dark navigation bar with links like 'Why GitHub?', 'Enterprise', 'Explore', 'Marketplace', and 'Pricing'. On the right of this bar are a search box, 'Sign in', and 'Sign up' buttons. The main heading is 'Create your account'. Below it are three input fields: 'Username *', 'Email address *', and 'Password *'. The password field has a note: 'Make sure it's at least 15 characters OR at least 8 characters including a number and a lowercase letter. [Learn more](#)'. Below these is the 'Email preferences' section with a checkbox 'Send me occasional product updates, announcements, and offers.' followed by a 'Verify your account' section. This section contains a CAPTCHA with the Russian text 'Решите эту задачу, чтобы мы знали, что вы реальный человек' and a 'Подтвердить' button. At the bottom is a blue 'Select a plan' button. A small footer note states: 'By creating an account, you agree to the Terms of Service. For more information about GitHub's privacy practices, see the GitHub Privacy Statement. We'll occasionally send you account-related emails.'

Первый шаг регистрации профиля на стартовой странице GitHub.

- После заполнения данных и успешного прохождения верификации нажимаем на кнопку Select a plan.

Why GitHub?EnterpriseExploreMarketplacePricing

Search GitHubSign InSign up

Join GitHub

Create your account

Username *

goshacademy

Email address *

go@htmlacademy.ru

Password *

Make sure it's at least 10 characters OR at least 8 characters including a number and a lowercase letter. [Learn more](#)

Email preferences

☐ Send me occasional product updates, announcements, and offers.

Verify your account



?

Select a plan

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

Второй шаг регистрации профиля на стартовой странице GitHub.

- Третий шаг — небольшой опрос от GitHub, который вы можете пройти, заполнив все поля и нажать Submit или пропустить, нажав skip this step.

Search or jump to...Pull requestsIssuesMarketplaceExplore

Selected plan: Free

Welcome to GitHub

Woohoo! You've joined millions of developers who are doing their best work on GitHub. Tell us what you're interested in. We'll help you get there.

What kind of work do you do, mainly?

Software EngineerSoftware Engineer

StudentStudent

Product ManagerProduct Manager

UX & DesignUX & Design

Data & AnalyticsData & Analytics

Marketing & SalesMarketing & Sales

TeacherTeacher

OtherOther

How much programming experience do you have?

NoneI don't program at all

A littleI'm new to programming

A moderate amountI'm somewhat experienced

A lotI'm very experienced

What do you plan to use GitHub for?

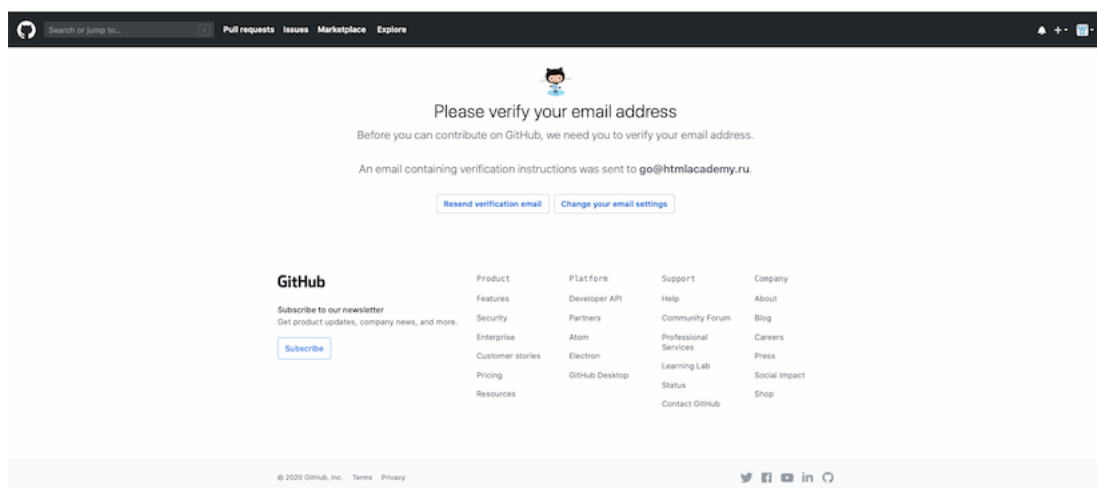
(Select up to 3)

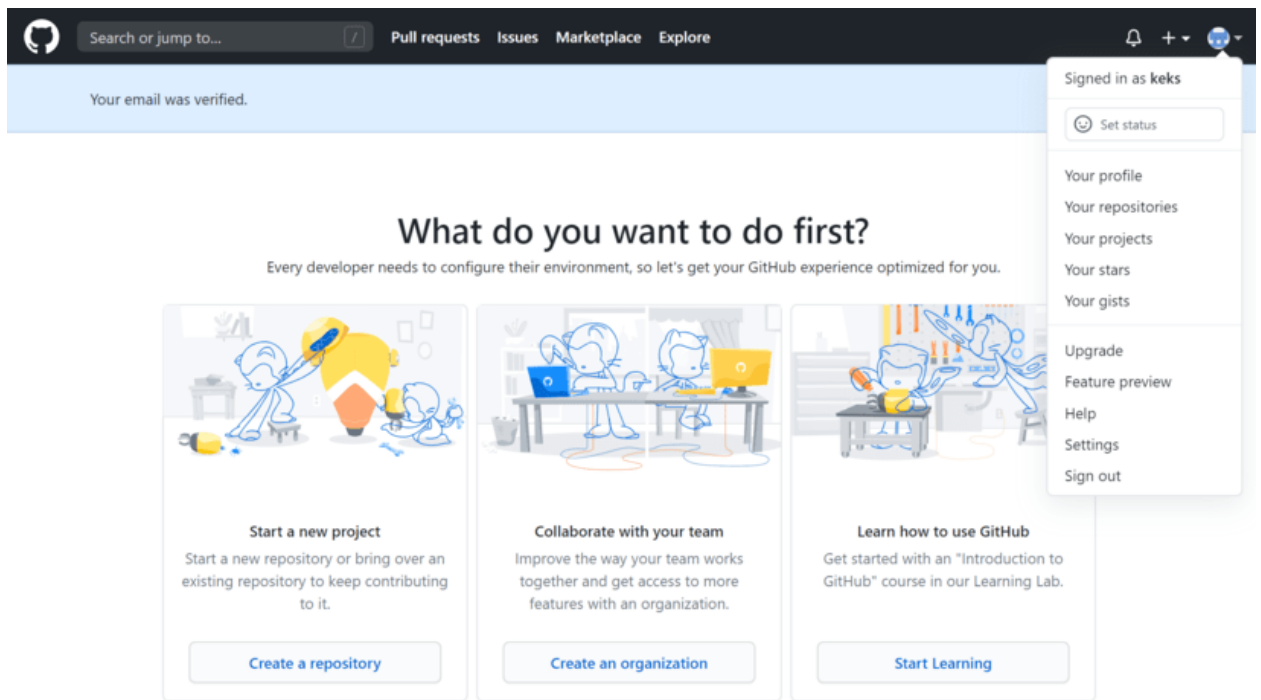
Опрос на третьем шаге регистрации.

2. После прохождения всех этапов на сайте, на указанный при регистрации ящик вам придет письмо от GitHub. Откройте его и подтвердите свой почтовый адрес, нажав **Verify email address** (подтвердить электронный адрес) или скопируйте вспомогательную ссылку из письма и вставьте её в адресную строку браузера.

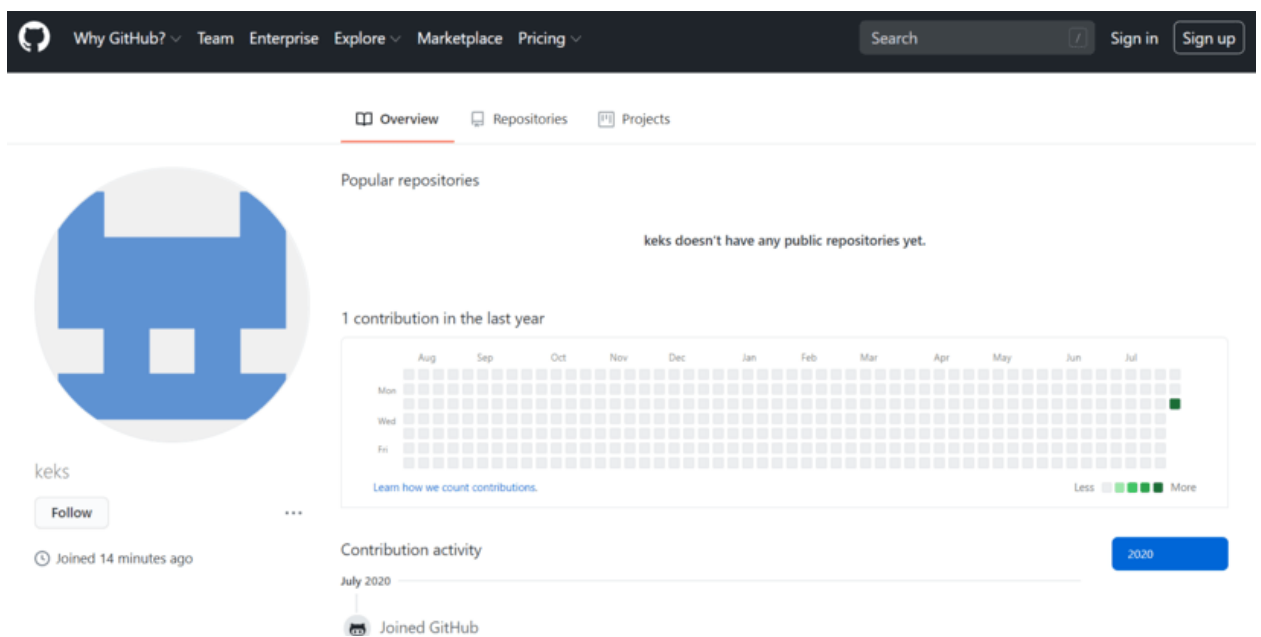
Подтверждение электронного адреса.

3. После верификации GitHub предложит создать новый репозиторий, организацию или узнать больше о GitHub. Этот пункт пока можно пропустить и перейти в профиль.





Переход в ваш профиль.



Так выглядит ваш профиль после регистрации.

Теперь у вас есть профиль на GitHub.

Шаг 2. Создание SSH-ключа для авторизации

Что такое **SSH-ключ** и зачем он нужен?

Чтобы работать со своего компьютера с GitHub, иметь доступ к проектам, хранящимся на сервисе, выполнять команды в консоли без постоянного подтверждения пароля, нужно пройти авторизацию у сервера. В этом помогают SSH-ключи.

Каждый SSH-ключ содержит пару: открытый (публичный) и закрытый (приватный) ключ. Открытый ключ отправляется на сервер, его можно не прятать от всех и не переживать, что кто-то его увидит и украдёт. Он бесполезен без своей пары — закрытого ключа. А вот закрытый ключ — секретная часть. Доступ к нему должен быть только у вас.

Вы отправляете какую-то информацию на сервер, где хранится ваш публичный ключ, сервер понимает, что вы это вы, то есть идентифицирует именно вас, и даёт вам какой-то ответ. И только вы можете расшифровать этот ответ, потому что только у вас есть подходящий закрытый ключ. Получается что-то вроде связки логин-пароль только намного безопасней. Ваш пароль кто-то может узнать или подобрать, а чтобы получить ваш приватный SSH-ключ, злоумышленнику придётся взломать ваш компьютер.

Чтобы пройти авторизацию по SSH-ключу, его надо сгенерировать или найти уже ранее созданный ключ на своём компьютере.

Сначала проверим, есть ли уже на компьютере ключ. По умолчанию SSH-ключи хранятся в каталоге `~/.ssh`, поэтому нужно проверить содержимое этого каталога.

1.

```
Evgenia repositorii % cd ~/.ssh
```

Открываем консоль и вводим команду:

```
ssh-keygen -t rsa -b 4096 -C "your_mail@example.com"
```

```
Evgenia .ssh % ssh-keygen -t rsa -b 4096 -C "michaylova1996@mail.ru"
```

- Далее нужно указать расположение файла для сохранения ключа. Если вы не введёте путь до файла и просто нажмёте Enter, ключ сохранится в файле, указанном в скобках.
- Теперь нужно установить пароль к вашему ключу и дважды ввести его. Если вы не хотите вводить пароль каждый раз, когда используете ключ, пропустите этот шаг, нажав «Enter», и ничего не вводите.

В результате выполнения получаем:

```
Your identification has been saved in /Users/xkaprizx/.ssh/id_rsa.
Your public key has been saved in /Users/xkaprizx/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:0lDVGVBaferd+2UfLC42PD8U9QmhUM2BsHyechZYIdg michaylova1996@mail.ru
The key's randomart image is:
+---[RSA 4096]---+
|      +==*00+      |
|      o.E*=+...    |
|      .  + + 000    |
|      o  o o....    |
|      . S. = ....   |
|      .  + .o o     |
|      .  . o+       |
|      *...o+        |
|      . =o. +       |
+-----[SHA256]-----+
```

Добавляем ключ в ssh-agent (сгенерированный или уже существующий).

```
-Evgenia .ssh % ssh-agent
```

Результат выполнения:

```
SSH_AUTH_SOCK=/var/folders/7_/hh7_wr1s33dbjqdyq4czdz140000gn/T//ssh-yktDdYxkHVsk/agent.62437; export SSH_AUTH_SOCK;
SSH_AGENT_PID=62438; export SSH_AGENT_PID;
echo Agent pid 62438;
```

Проверяем доступность ключа командой `eval "$(ssh-agent -s)"`

```
-Evgenia .ssh % eval "$(ssh-agent -s)"
```

Получаем следующий результат:

```
Agent pid 62440
```

Добавляем с помощью `ssh-add ~/.ssh/your_key_name`, где указываем верный путь до файла с ключом и его имя.

```
-Evgenia .ssh % ssh-add ~/.ssh/id_rsa
```

Полученный результат:

```
Identity added: /Users/xkaprizx/.ssh/id_rsa (michaylova1996@mail.ru)
```

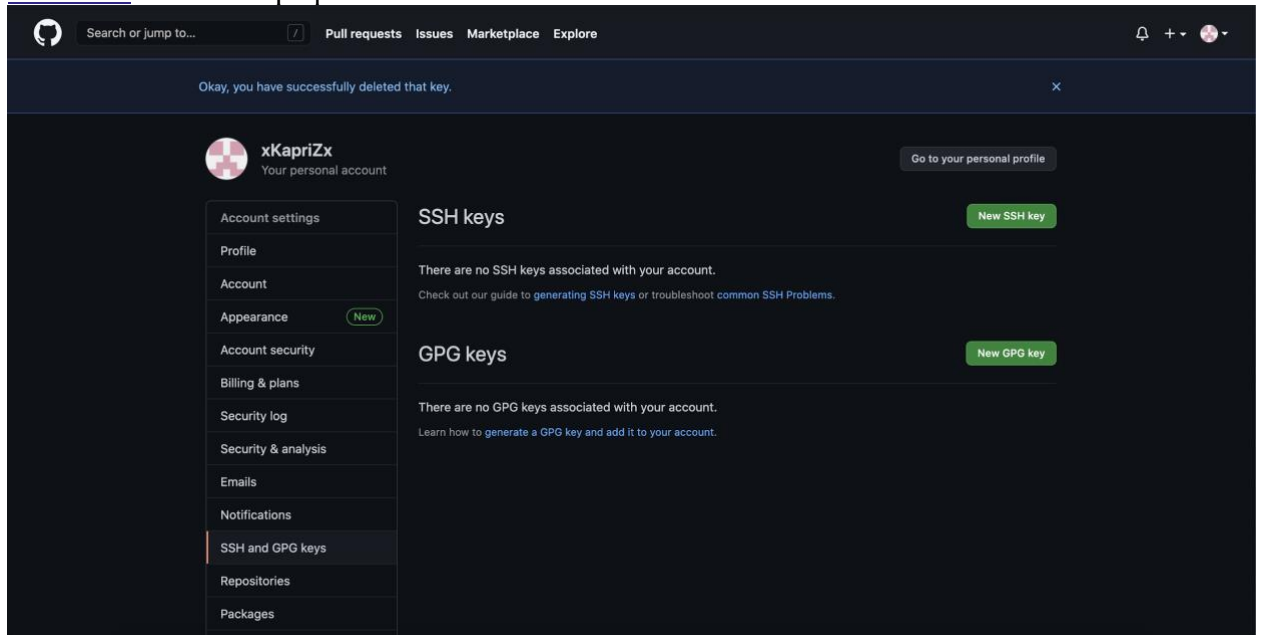
Чтобы связать свой локальный и удаленный репозиторий, выведите в консоль содержимое файла с помощью команды `cat`:

```
-Evgenia .ssh % cat ~/.ssh/id_rsa.pub
```

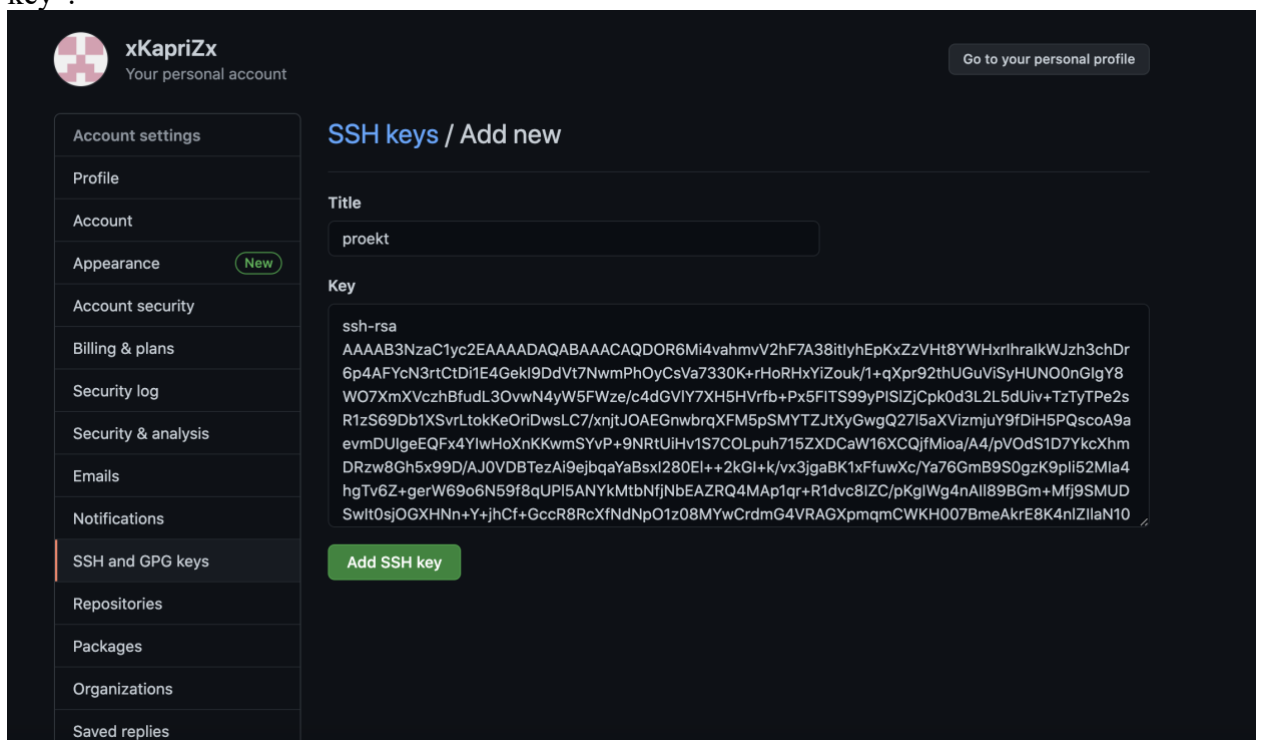
В итоге мы должны получить вот такой ключ (у вас будет другой набор символов):

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCDOR6Mi4vahmvV2hF7A38itIyEpKxZzVht8YWHxrihraIkwJzh3chDr6p4AFYcN3rtCtDi1E4GekI9DdVt7NwmPh
OyCsVa7330K+rHoRHxYiZouk/1+qXpr92thUGuViSyHUN00nGIgY8W07XmXVczhBfudL30vwn4yW5FWze/c4dGV1Y7XH5HVrfb+Px5FITS99yPIS1ZjCpk0d3L2L5d
Uiv+TzTyTPe2sR1zS69Db1XSvrltokKeOriDwsLC7/xnjtJOAEGnwbrqXFM5pSMYTZJtXyGwgQ2715aXVizmjuY9fDiH5PQscoA9aevmDUlgeEQF4YIwHoXnKKwmS
YvP+9NRtUiHv1S7COLpuh715ZXDcAw16XCQjfmioa/A4/pV0dS1D7YkcXhmDRzw8Gh5x99D/AJ0VDBTezAi9ejbqaYaBsXI280E1++2kGI+k/vx3jgaBK1xFuwXc/
Ya76GmB9S0gzK9pIi52MIa4hgTv6Z+gerW69o6N59f8qUP15ANYkmtbNfjNbEAZRQ4MAp1qr+R1dvc8IZC/pKGIWg4nA1189BGm+Mfj9SMUDSwIt0sj0GXHNn+Y+jh
Cf+GccR8RcXfNdNp01z08MYwCrdmG4VRAGXpmqmCWKH007BmeAkrE8K4nLZI1aN10+3hMMq/8Sk1mWxSMgn89hIyA9bQ== michaylova1996@mail.ru
```

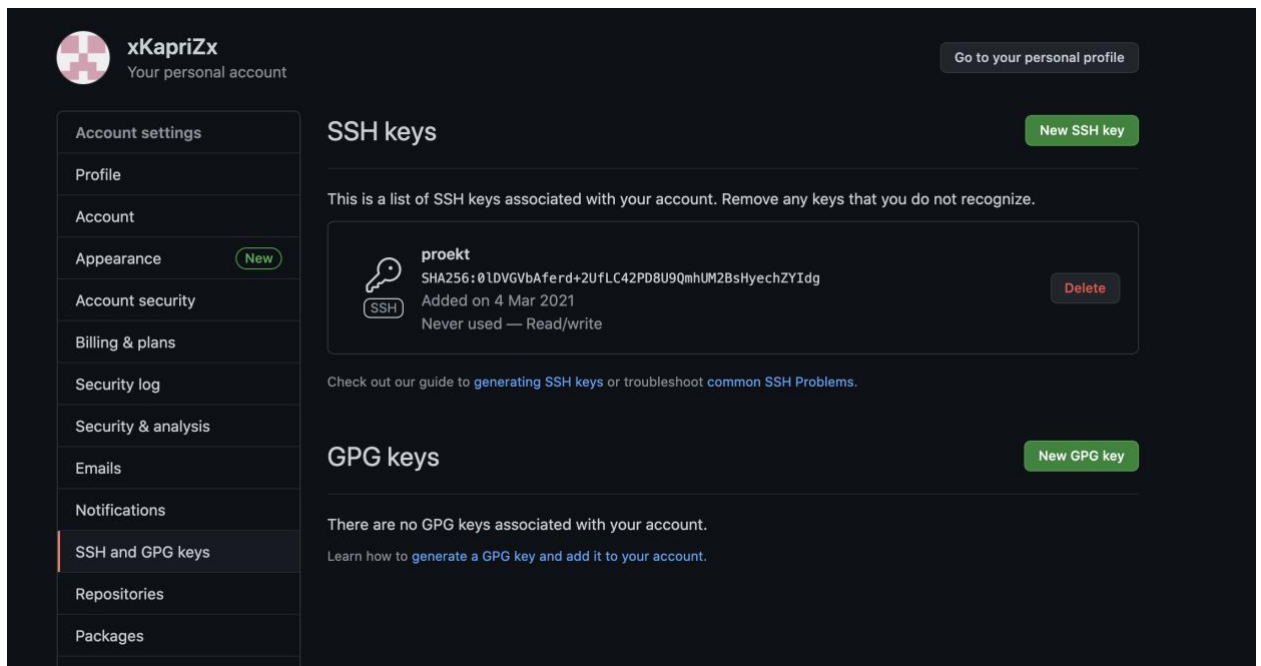
Мы должны его скопировать из консоли, перейти на [страницу для работы с ключами](#) в вашем профиле на GitHub.



Выбираем кнопку “New SSH key”, открывается окно с вводом данных, в поле “key” вставляем скопированный ключ, в “Title” вводим любое имя ключа и нажимаем “Add SSH key”.



Вот так выглядит добавленный ключ:



Шаг 3. Создание нового репозитория для своего проекта

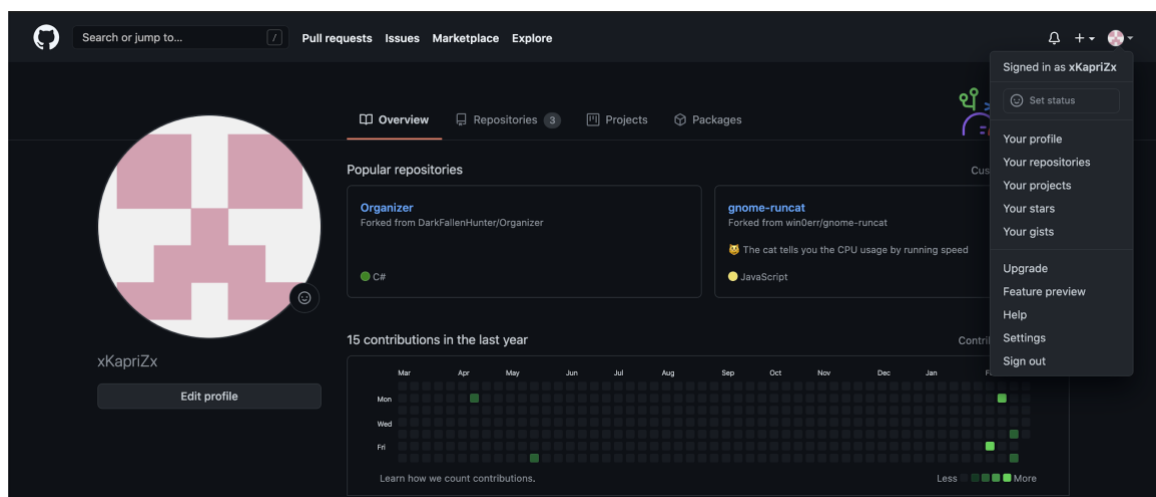
Примечание. Репозиторий создается в отдельной папке, не вложенной в репозиторий из прошлых шагов.

1. Для начала необходимо перейти в другую директорию и создать папку проекта (Часть 1, шаг 3, пункт 1-3).
2. Создайте несколько файлов (например, файл readme.txt и какие-нибудь файлы для кода), чтобы их создать необходимо повторить следующий шаг (часть 1, шаг 3, пункт 4).
3. Создайте новый локальный репозиторий (Часть 1, шаг 4).

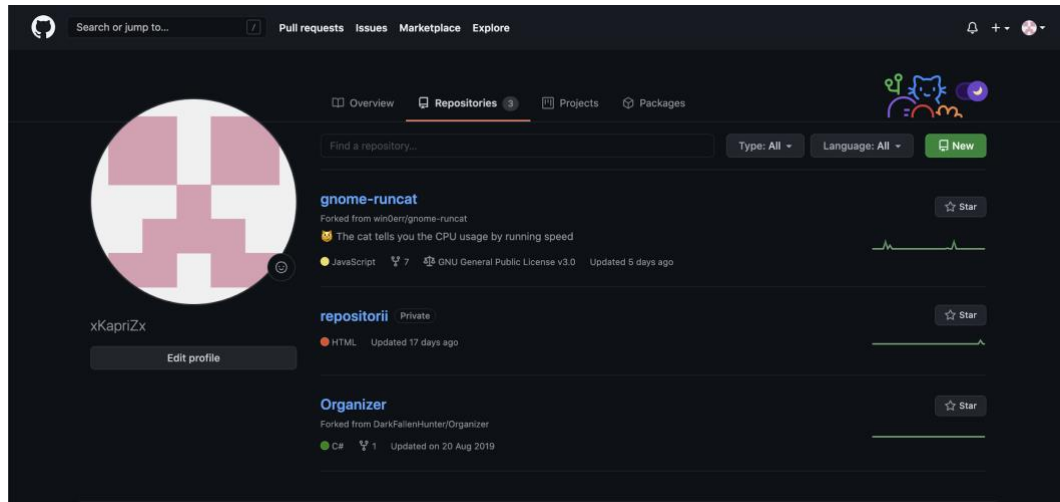
Шаг 4. Связываем локальный и удаленный репозитории

Для того, чтобы связать репозиторий на локальной машине и удаленный репозиторий, необходимо проделать следующие действия:

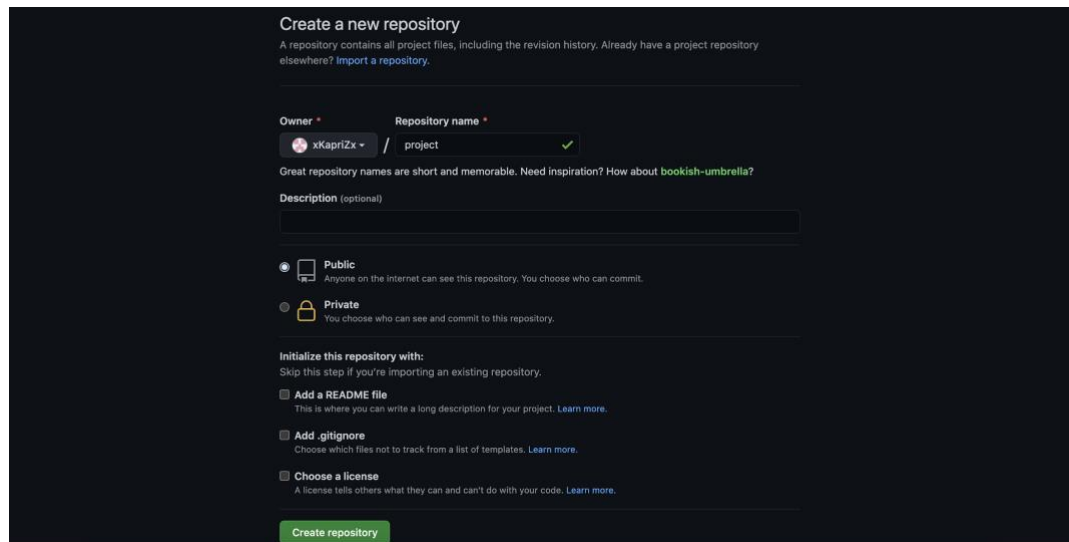
1. Заходим на GitHub, на свою страницу, выбрать вкладку репозитории (можно на главной странице, можно в правом верхнем углу)



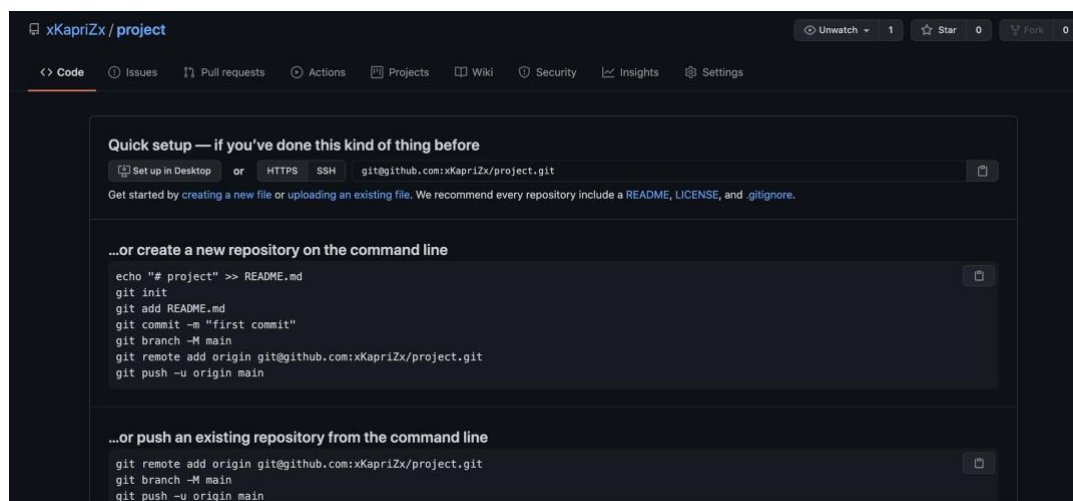
2. На вкладке с репозиториями, нажимаем на зеленую кнопку “New”



3. Называем наш репозиторий и выбираем приватность репозитория (публичный или приватный)



4. После проделанных шагов, мы создали пустой репозиторий на GitHub, осталось его только связать с локальным репозиторием



5. Чтобы связать репозитории друг с другом необходимо ввести в консоль следующую команду:

```
git remote add project git@github.com:<ваши имя и название репозитория>.git
```

```
-Evgenia repositorii % git remote add project git@github.com:xKapriZx/project.git
```

Готово, мы связали наши репозитории между собой.

Шаг 5. Создание веток и переключение между ними

Для того, чтобы создать ветку, необходимо ввести команду **git branch <branch>**, где вместо <branch>, вы вводите название своей ветки (например, git branch dev создаст ветку с именем dev).

Чтобы переключиться на другую ветку нужно ввести команду **git checkout <branch>**, где вместо <branch>, вы вводите название своей ветки (например, git checkout dev переключит на ветку с именем dev).

1. Для создания новой ветки необходимо воспользоваться командой **git checkout -b <имя ветки>**:

```
-Evgenia repositorii % git checkout -b main
```

```
-Evgenia repositorii % git status
```

```
На ветке main  
нечего коммитить, нет изменений в рабочем каталоге
```

2. Сделайте какие-либо изменения в файлах. (Часть 1, шаг 7, пункт 1)
3. Проиндексируйте изменения и закоммитьте.

Шаг 6. Слияние веток

Слияние производится с помощью команды **git merge <название ветки>**, где параметр указывает из какой ветки вы хотите слить изменения. Сначала вам может быть сложно понять, как пользоваться этой командой, но давайте разберём пример, чтобы стало понятнее.

Представим ситуацию, когда мы находимся в ветке branch2 и нам надо произвести слияние из ветки **branch1** в **branch2** (именно в таком порядке). То есть нам нужно, чтобы все изменения из ветки branch1 оказались и в ветке branch2. Тогда для этого мы должны, **находясь в ветке branch2**, ввести команду **git merge branch1**. В этом случае мы выполним поставленную задачу.

Представим еще одну ситуацию, когда мы в 2 ветках редактировали один и тот же файл, в этом случае возникнет конфликт и команда выдаст ошибку, и как его разрешить [см. Разрешение конфликтов при слиянии](#).

1. Для того, чтобы слить ветки необходимо воспользоваться командой *git merge* *<название ветки>*:

```
Evgenia repositorii % git merge main
```

Результат выполнения данного слияния:

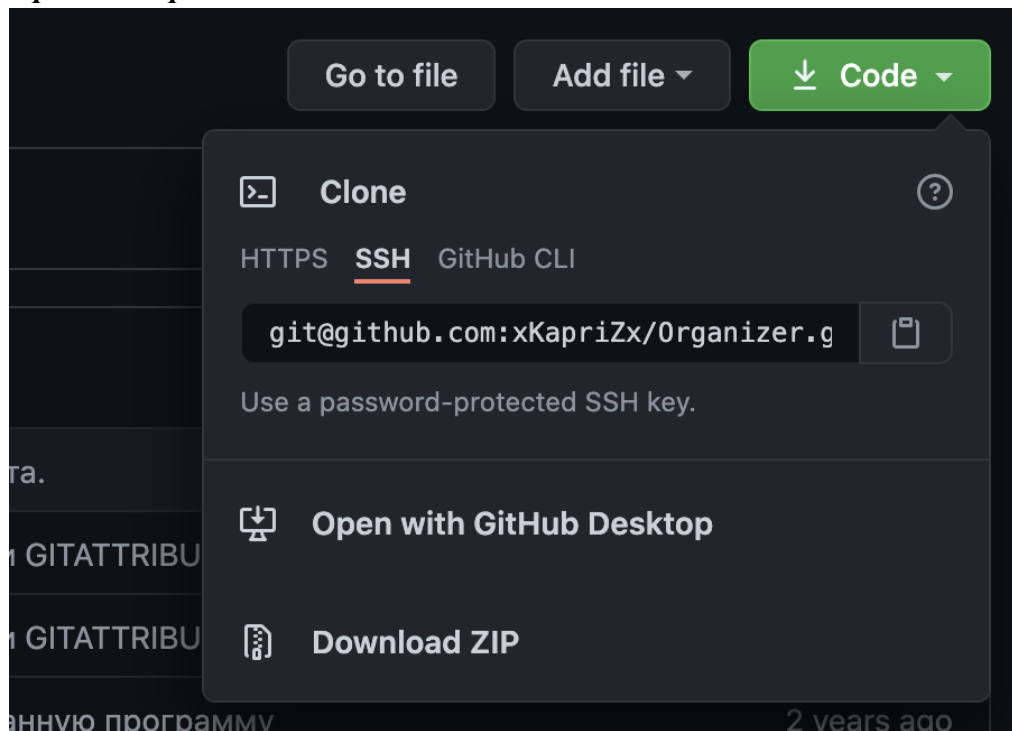
```
Обновление de0643d..42ca495
Fast-forward
 proekt.html | 1 +
 1 file changed, 1 insertion(+)
```

Дополнительная информация для выполнения заданий по вариантам

1. Клонирование удаленного репозитория

Клонирование репозитория можно выполнить при помощи команды **git clone <ссылка на репозиторий>**. В качестве репозитория можно использовать любой публичный репозиторий. Ссылку для клонирования можно получить на странице самого репозитория по выпадающему меню Code.

Примечание: клонировать репозиторий необходимо в отдельный каталог, вне текущего репозитория.



1. Для клонирования репозитория воспользуемся указанной командой **git clone <ссылка на репозиторий>**:

```
Evgenia praktika % git clone git@github.com:xKapriZx/0rganizer.git
```

В результате мы получим следующее сообщение:

```
Клонирование в «Organizer»...
The authenticity of host 'github.com (140.82.121.4)' can't be established.
RSA key fingerprint is SHA256:nThbg6kXUpJWGl7E1IGOCspRomTxdCARLviKw6E5SY8.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com,140.82.121.4' (RSA) to the list of known hosts.
remote: Enumerating objects: 38, done.
remote: Counting objects: 100% (38/38), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 38 (delta 7), reused 35 (delta 6), pack-reused 0
Получение объектов: 100% (38/38), 33.12 KiB | 289.00 KiB/s, готово.
Определение изменений: 100% (7/7), готово.
```

2. Выгрузка изменений в удаленный репозиторий

Выгрузка в удаленный репозиторий выполняется с помощью команды **git push origin <branch>**, где вместо branch вы пишете имя ветки, в который хотите выгрузить на гитхаб.

Например, **git push origin branch1** выгрузит данные из ветки branch1 локального репозитория в удаленный репозиторий (в нашем случае из компьютера в гитхаб).

3. Вывод в консоль различий между веткой мастер и другой веткой

Вывод различий между несколькими вариантами состояниями репозитория, в том числе между несколькими ветками, можно вывести при помощи команды **git diff**. Для вывода различий между двумя ветками можно использовать данную команду в следующем варианте **git diff <branch1> <branch2>**. В результате будет выведен список всех различий между двумя указанными ветками.

1. Воспользуемся указанной командой для вывода различий между репозиториями **master** и **newbranch**:

```
Evgenia repositorii % git diff master newbranch
```

В результате мы получим следующее сообщение:

```
diff --git a/.DS_Store b/.DS_Store
deleted file mode 100644
index 2e7412b..0000000
Binary files a/.DS_Store and /dev/null differ
diff --git a/proekt.html b/proekt.html
index 99392c8..5042bec 100644
--- a/proekt.html
+++ b/proekt.html
@@ -1,7 +1,4 @@
<html>
-<head>
-<h2>Текст для слияния веток</h2>
-</head>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
</body>
```

4. Создание тэгов к коммитам

Тэги необходимы для идентификации коммитов. Можно использовать их для указания коммита, на котором была завершена разработка beta-версии проекта. Управлять тэгами к коммитам можно при помощи команды **git tag**.

1. Перейдем к коммиту первой версии при помощи команды **git checkout <хэш коммита>**
2. Создадим тег первой версии **git tag v1**

```
ro-Magerram git_tutorial % git tag v1
ro-Magerram git_tutorial %
```

3. Давайте создадим тег для версии, которая идет перед текущей версией и назовем его v1-beta. В первую очередь нам надо переключиться на предыдущую версию. Вместо поиска по хэшу, мы будем использовать ^, обозначающее «родитель v1».

git checkout v1^

cat proekt.html

```
vgtstptlk@MacBook-Pro-Magerram git_tutorial % git checkout v1^
Note: switching to 'v1^'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at bbee410 Added standart HTML page tags
```

Это версия с тегами <html> и <body>, но еще пока без <head>. Давайте сделаем ее версией v1-beta

git tag v1-beta

```
vgtstptlk@MacBook-Pro-Magerram git_tutorial % git tag v1-beta
vgtstptlk@MacBook-Pro-Magerram git_tutorial %
```

4. Попробуем переключиться между метками

git checkout v1

```
vgtstptlk@MacBook-Pro-Magerram git_tutorial % git checkout v1
Previous HEAD position was bbee410 Added standart HTML page tags
HEAD is now at c993c20 pervii commit na GitHub
```

git checkout v1

```
vgtstptlk@MacBook-Pro-Magerram git_tutorial % git checkout v1-beta
Previous HEAD position was c993c20 pervii commit na GitHub
HEAD is now at bbee410 Added standart HTML page tags
```

5. Просмотр тэгов с помощью команды tag. Давайте рассмотрим наши тэги с помощью команды **git tag**

```
HEAD is now at b80c410 Added standard HTML page tags
[vgtstptlk@MacBook-Pro-Magerram git_tutorial % git tag
v1
v1-beta
```

5. Работа с отложенными изменениями - git stash

Git имеет возможность отложить изменения, записав их в специальное хранилище. Сделать это можно при помощи команды **git stash**. При использовании данной команды изменения в файлах записываются в условный “архив”, который будет хранить их без индексирования и коммита. Такой функционал может пригодиться, если есть необходимость перейти к работе с другой веткой

1. Внесем изменения в файл proekt.html.

```
[xkaprizx@MBP-Evgenia repositorii % cat proekt.html
<html>
<head>
<h2>Текст для слияния веток</h2>
</head>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
<p>Git Stash</p>
</body>
</html>
```

2. Запомним эти изменения при помощи **git stash**.

```
Evgenia repositorii % git stash
```

В результате мы получим следующее сообщение:

```
Рабочий каталог и состояние индекса сохранены WIP on master: 42ca495 для слияния
```

После этого изменения запишутся в архив и пропадут из текущей версии.

```
[xkaprizx@MBP-Evgenia repositorii % cat proekt.html
<html>
<head>
<h2>Текст для слияния веток</h2>
</head>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
</body>
</html>
```

3. Добавим изменения, сохраненные при помощи **git stash** в ветку newbranch. Для этого необходимо воспользоваться командой **git stash pop**.

```
[xkaprizx@MBP-Evgenia repositorii % git checkout newbranch
Переключено на ветку «newbranch»
[xkaprizx@MBP-Evgenia repositorii % git stash pop
```

В результате мы получим следующее сообщение:


```
Автослияние proekt.html
На ветке newbranch
Изменения, которые не в индексе для коммита:
  (используйте «git add <файл>...», чтобы добавить файл в индекс)
  (use "git restore <file>..." to discard changes in working directory)
      изменено:      proekt.html

нет изменений добавленных для коммита
(используйте «git add» и/или «git commit -a»)
Отброшено refs/stash@{0} (4c41bd5db61b76e71ebb1810800411edde5035d1)
```

А содержимое файла сменится на следующее:

```
<html>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
<p>Git Stash</p>
</body>
</html>
```

6. Перемещение коммита в другую ветку

Перемещать коммиты между ветками можно при помощи команды **git cherry-pick**. Данный функционал можно использовать, если нет необходимости сливать всю ветку целиком, а достаточно взять из нее несколько изменений.

1. Для начала добавим изменение в файл proekt.html.

```
<html>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
<p>Нововведение для ветки мастер</p>
</body>
</html>
```

2. Проиндексируем и закоммитим данное изменение
3. При помощи команды git log узнаем хэш сделанного коммита.

```
commit 8b4f872e4cd9837d2b430aad082b64c9107dfb3d (HEAD -> newbranch)
Author: Евгения <michaylova1996@mail.ru>
Date:   Fri Mar 5 19:11:21 2021 +0300
```

Коммит для cherry-pick

4. Перейдем в ветку master - git checkout master
5. Выполним команду git cherry-pick для добавления последнего коммита из ветки newbranch.

```
Evgenia repositorii % git cherry-pick 8b4f872e4cd9837d2b430aad082b64c9107dfb3d
```

В результате мы получим следующее сообщение:

```
Автослияние proekt.html
[master 0267ffa] Коммит для cherry-pick
Date: Fri Mar 5 19:11:21 2021 +0300
1 file changed, 1 insertion(+)
```

7. Добавление изменений в последний коммит

Внести изменения в последний коммит, не создавая новый, можно при помощи команды git commit --amend.

1. Добавим в репозиторий readme.md файл при помощи редактора nano.

```
GNU nano 2.0.6 File: readme.md
#Репозиторий для практической работы №1 по дисциплине ТРПП
Данный репозиторий содержит файлы для работы с git
```

2. Теперь добавим файл readme.md в последний коммит при помощи команды `git commit --amend`.

После вызова данной команды откроется текстовый редактор для переименования коммита.

Примечание: для добавления файлов в коммин без изменения комментария необходимо добавить флаг `--no-edit`.

```
GNU nano 2.0.6 File: /Users/xkaprizx/Documents/repositorii/.git/COMMIT_EDITMSG
Коммит для cherry-pick
# Пожалуйста, введите сообщение коммита для ваших изменений. Строки,
# начинающиеся с «#» будут проигнорированы, а пустое сообщение
# отменяет процесс коммита.
#
# Дата:      Fri Mar 5 19:11:21 2021 +0300
#
# На ветке master
# Изменения, которые будут включены в коммит:
#       изменено:      proekt.html
#
# Неотслеживаемые файлы:
#       readme.md
#
```

После сохранения и закрытия текстового редактора получим следующее сообщение:

```
[master cf782cb] Добавление файла readme.md
Date: Fri Mar 5 19:11:21 2021 +0300
1 file changed, 1 insertion(+)
```

3. Проверим комментарий коммита при помощи `git log`.

```
commit cf782cb28b3104fa229b7a34694541bd87e7d539 (HEAD -> master)
Author: Евгения <michaylova1996@mail.ru>
Date:   Fri Mar 5 19:11:21 2021 +0300

    Добавление файла readme.md
```

Как видно из скриншота комментарий был изменен, а значит, изменения успешно записались.

8. Вывод изменений строк файла по коммитам

Существует несколько вариантов того, как можно вывести информацию об изменениях строк файла. Первый вариант - использование команды `git blame <имя файла>`. Данная команда выводит для каждой строки файла пометку того, кем и в каком коммите она была изменена.

1. Выведем информацию по изменениям в файле при помощи команда `git blame <filename>`

```
[xkaprizx@MBP-Evgenia repositorii % git blame proekt.html
```

В результате мы получим следующее сообщение:

```
bbee4102 (Evgenia 2021-02-12 10:40:30 +0300 1) <html>
c993c20b (Евгения 2021-02-15 21:58:08 +0300 2) <head>
42ca4955 (Евгения 2021-03-04 18:42:06 +0300 3) <h2>Текст для слияния веток</h2>
c993c20b (Евгения 2021-02-15 21:58:08 +0300 4) </head>
bbee4102 (Evgenia 2021-02-12 10:40:30 +0300 5) <body>
8db4b7f9 (Evgenia 2021-02-12 10:14:43 +0300 6) <h1>Мы создали своё наполнение в данном файле!</h1>
cf782cb2 (Евгения 2021-03-05 19:11:21 +0300 7) <p>Нововведение для ветки мастер</p>
bbee4102 (Evgenia 2021-02-12 10:40:30 +0300 8) </body>
bbee4102 (Evgenia 2021-02-12 10:40:30 +0300 9) </html>
```

Второй вариант - использование команды **git log -p <имя файла>**. Она выводит подробную информацию по изменениям конкретного файла.

1. Выполним команду **git log -p <filename>** для того же файла - projekt.html.

```
Evgenia repositorii % git log -p projekt.html
```

В результате мы получим сообщение для каждого изменения в коммитах. Приведем часть вывода:

```
commit cf782cb28b3104fa229b7a34694541bd87e7d539 (HEAD -> master)
Author: Евгения <michaylova1996@mail.ru>
Date: Fri Mar 5 19:11:21 2021 +0300

    Добавление файла readme.md

diff --git a/projekt.html b/projekt.html
index 99392c8..b97be9c 100644
--- a/projekt.html
+++ b/projekt.html
@@ -4,5 +4,6 @@
</head>
<body>
<h1>Мы создали своё наполнение в данном файле!</h1>
+<p>Нововведение для ветки мастер</p>
</body>
</html>

commit 42ca4955b7150edc898854d79190699366f798b7 (main)
Author: Евгения <michaylova1996@mail.ru>
Date: Thu Mar 4 18:42:06 2021 +0300

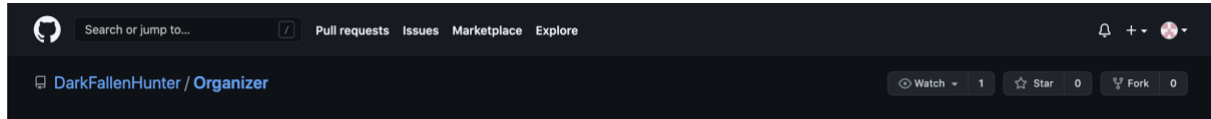
    для слияния

diff --git a/projekt.html b/projekt.html
index 9954b62..99392c8 100644
--- a/projekt.html
+++ b/projekt.html
@@ -1,5 +1,6 @@
<html>
<head>
```

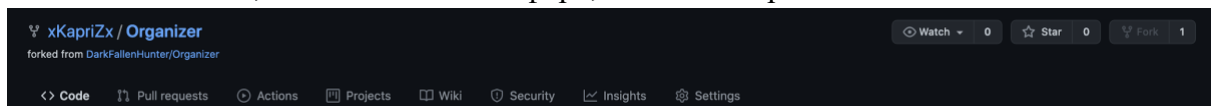

Часть 3. Ветвление и оформление кода

Шаг 1. Форк репозитория

1. Заходим на GitHub под своим логином и паролем.
2. Переходим по ссылке в репозиторий, который собираемся форкать.
3. В верхнем правом углу нажимаем кнопку “Fork”.



После того, как Вы нажали на форк, вместо автора появляется на его месте Вы.



Теперь у Вас есть точная копия оригинального репозитория.

Шаг 2. Клонирование репозитория на локальную машину

1. Перейдите в другую папку, не вложенную в репозиторий
2. Склонируйте репозиторий в эту папку (Часть 2. Доп. информация. Пункт 1)

Шаг 3. Создание двух веток в master'e

Для выполнения данного задания (Часть 2. Шаг 5)

Шаг 4. 3 коммита в каждую из веток, которые меняют один и тот же кусочек файла

(Часть 1. Шаг 5)

Шаг 5. Слияние ветки branch1 в ветку branch2 с разрешением конфликтов

1. Слейте 2 ветки, используя команду **git merge** (Часть 2. Шаг 6)
2. Разрешите конфликты при слиянии

Существует несколько вариантов разрешения конфликтов:

- 1) Вручную разрешить конфликты

После merge в файле появятся куски из обеих сливаемых версий:

<<<<<<< HEAD

содержимое файла из первой ветки

=====

содержимое файла из второй ветки

>>>>>>> otherbranch

Нужно будет вручную отредактировать конфликтный файл (или файлы), при этом не забывая удалить метки, оставленные Git (**>>>>>>> otherbranch**). Затем ввести команды:

git add conflicting-file-name.txt

git commit -m'merged A and B'

2) Выбрать одну из версий файла

Можно явным образом указать: какой файл выбирать. Подходит, только если одна из версий не нужна.

git checkout --ours a.txt

git checkout --theirs a.txt

git add a.txt

git commit -m "added theirs"

Для решения конфликта попробуйте воспользоваться командой **git mergetool**

Более подробно о решении конфликтов можно прочитать тут:

<https://webdevkin.ru/courses/git/git-conflicts>

<https://www.atlassian.com/ru/git/tutorials/using-branches/merge-conflicts>

[https://git-](https://git-scm.com/book/ru/v2/%D0%92%D0%B5%D1%82%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D0%B5-%D0%B2-Git-%D0%9E%D1%81%D0%BD%D0%BE%D0%B2%D1%8B-%D0%B2%D0%B5%D1%82%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D1%8F-%D0%B8-%D1%81%D0%BB%D0%B8%D1%8F%D0%BD%D0%B8%D1%8F)

[scm.com/book/ru/v2/%D0%92%D0%B5%D1%82%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D0%B5-%D0%B2-Git-%D0%9E%D1%81%D0%BD%D0%BE%D0%B2%D1%8B-%D0%B2%D0%B5%D1%82%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D1%8F-%D0%B8-%D1%81%D0%BB%D0%B8%D1%8F%D0%BD%D0%B8%D1%8F](https://git-scm.com/book/ru/v2/%D0%92%D0%B5%D1%82%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D0%B5-%D0%B2-Git-%D0%9E%D1%81%D0%BD%D0%BE%D0%B2%D1%8B-%D0%B2%D0%B5%D1%82%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D1%8F-%D0%B8-%D1%81%D0%BB%D0%B8%D1%8F%D0%BD%D0%B8%D1%8F)

Шаг 6. Выгрузка всех изменений во всех ветках в удаленный репозиторий

1. Для перехода по веткам используйте команду **git checkout** (Часть 2. Шаг 5)
2. Для выгрузки изменений используйте команду **git push** (Часть 2. Доп. информация. Раздел 2)

Шаг 7. Выполнение 3-х коммитов в branch1 (Часть 1. Шаг 5)

Шаг 8. Повторное клонирование репозитория в другую директорию

1. Перейдите в другую папку, не вложенную в репозиторий
2. Склонируйте репозиторий в эту папку (Часть 2. Доп. информация. Пункт 1)

Шаг 9. Выполнение 3-х коммитов в ветку branch1 (Часть 1. Шаг 5)

Шаг 10. Выгрузка всех изменений в удаленный репозиторий

Для выгрузки изменений используйте команду **git push** (Часть 2. Доп. информация. Раздел 2)

Шаг 11. Выгрузка изменения из старого репозитория с опцией --force

Для выгрузки изменений используйте команду **git push** (Часть 2. Доп. информация. Раздел 2)

Опция --force позволяет произвести принудительную выгрузку в репозиторий, даже если в нем обнаружены конфликты слияния.

Шаг 12. Получение всех изменения в новом репозитории

Получение изменений из удаленного репозитория выполняется командой **git pull**

Примечания

Где находится файл .gitconfig в macOS?

Для macOS не нужно искать файл .gitconfig

Необходимо воспользоваться следующей командой:

```
git config --global --edit
```

Что делать, если возникает ошибка при git init?

Если возникает ошибка:

```
git init-неверное числовое значение конфигурации 'auto' для 'core.autocrlf' in
```

То ввести:

```
git config --global core.autocrlf false
```

Контрольные вопросы

1. Какие существуют типы систем контроля версий? Приведите примеры к каждому типу.
2. К какому типу систем контроля версий относится Git?
3. Что такое репозиторий Git?
4. Что такое коммит?
5. Что такое ветка в репозитории Git?
6. Что такое тег в репозитории Git?
7. Что такое слияние двух веток?
8. Что такое конфликт в Git? Как его решить и почему они бывают?
9. Как отменить слияние веток, если произошел конфликт?
10. Для чего нужен .gitignore?
11. Что делает команда git status?
12. Что делает команда git add?
13. Что делает команда git log?
14. Что делает команда git diff?
15. Что делает команда git show?
16. Что делает команда git stash?
17. Как добавить новую директорию в Git?
18. Что сделает команда "git branch" без какого-либо параметра?
19. Чем отличаются команды "git push" и "git pull"?
20. Что означает статус файла untracked в выводе команды git status?
21. Что означает статус файла new в выводе команды git status?
22. Что означает статус файла modified в выводе команды git status?
23. Чем отличается master и origin/master?
24. Как узнать, кто автор строчки в файле, используя систему Git?
25. Как отменить действие команды "git add" на файл?
26. Как привести измененный файл в начальное состояние (до изменения)?
27. Как сделать ветку с названием my_branch?
28. Как сделать коммит для ветки my_branch?
29. Как удалить локальную ветку my_branch?
30. Как исправить ошибку "fatal: The current branch my_branch has no upstream branch", возникающую при вводе git push?
31. Как удалить все untracked файлы?
32. Что такое GitHub?
33. Что такое форк репозитория?
34. Что такое pull request?

Требования к отчету

По итогу выполнения практической работы необходимо оформить отчёт, включающий:

1. Титульный лист;
2. Оглавление;
3. Формулировки заданий и скриншоты командной строки с требуемыми действиями для задач курсов <https://githowto.com/> (1 и 2 часть ПР);
4. Скриншоты действий, выполненных согласно варианту из частей 2 и 3 практической работы (не забудьте указать в отчете вариант и задание), а также скриншоты выполнения общего задания (п.7);
5. Ответы на контрольные вопросы (8 любых) с развернутым пояснением;
6. Вывод о проделанной работе.

Отчет о практической работы необходимо загрузить в СДО (в случае каких-либо технических проблем отчет необходимо выслать на почту преподавателя в домене @mirea.ru)